

Unsupervised Textless Prompt With LLMs for Multivariate Time Series Anomaly Detection

Guoxiang Zhong¹, Jie Liu², Fagui Liu³, *Member, IEEE*, Jun Jiang⁴, *Member, IEEE*, Bin Wang⁵,
Fan Zhang⁶, and C. L. Philip Chen⁷, *Life Fellow, IEEE*

Abstract—Prompting the pre-trained large language models (LLMs) presents a novel implement to detect multivariate time series anomalies, which demonstrates strong potentiality. However, adding a textual string or embedding to numerical sequence for modeling the prompt has remained elusive due to lacking of exact word descriptions and prior information about initial time series input. In this article, we design an unsupervised textless prompt to fine-tune the LLMs and further propose the LLM-based anomaly detector (LLM-AD). The prompt approach renovates the time-stamp masked operation to variate masked operation based on the observation that the root causes variates about anomalies destroy the normal variates imputation procedure. The unmasked and masked variates are respectively confirmed as prompt input and prompt answer, which gets rid of the text-driven prompt template. Minimizing the divergence between predicted and actual prompt answers fine-tunes the LLMs which only the feedforward and normalization layers are activated. During the next testing stage, an anomaly alert will be triggered if the output answer is made far enough away from actual masked variates. Extensive experiments compared with the nine baseline algorithms reveal that our LLM-AD achieves competitive detection results, boosting the averaged f1-score of four benchmark datasets from 85.28% to 91.62%.

Index Terms—Anomaly detection, large language models (LLMs), multivariate time series, prompt learning.

I. INTRODUCTION

MULTIVARIATE time series anomaly detection, one of the typical research-intensive points in time series analysis [1], [2], [3], [4], has been widely applied in various domains, such as data-center operation [5], space exploration [6], industrial control systems [7], [8], and so on. Traditional deep small models, including recurrent neural networks [9], convolution neural networks [10], graph neural networks [11], [12], and Transformers [13], [14], reach the performance bottleneck due to parameter size, although they had sequentially achieved the SOTA results in the past few years. Since ChatGPT¹ was first released to the public, we have witnessed the great development of pre-trained large language models (LLMs) in natural language processing (NLP) [15], [16], [17]. As a result, some people suggest that LLMs can also create a brand new land for the time series field in the foreseeable future [18], [19], [20], [21], [22] because both modeling text and time series data rely on considering the contextual dependency relationship.

The LLMs are pre-trained mainly on the pretext textual datasets and then predict the downstream task through fine-tuning with prompt [23]. Fig. 1(a) presents how to attain the classification label about initial input data through prompt learning. For the case of sentiment analysis in Fig. 1(b), the expert searches the prompt template “[] because it is [].” through comprehending the initial input sentence “I like this car.”. The prompt input appears as “I like this car because it is [].” The pre-trained LLMs answer the empty prompt slot (“good”) which can be mapped as the goal label (“++”). The reason why this approach is practical is that acquiring semantic information about text input comes naturally to most humans. The additional textual string in the prompt template aligns with the initial input sentence. However, the numerical input source data remains a significant impediment, while we can take advantage of LLMs and adapt these models for detecting time series anomalies. The text component of the prompt template always relies on the word descriptions about time series. Unlike the text, time series cannot be directly conveyed by words from the semantic view, as shown in Fig. 1(c). Moreover, anomaly detection is always in an unsupervised setting, which eradicates the operations with prior knowledge about breaking this dilemma.

In this article, we develop the unsupervised textless prompt to eliminate these obstacles and then propose LLM-based

Received 6 November 2024; revised 18 August 2025; accepted 25 December 2025. This work was supported in part by the Major Key Project of PCL under Grant PCL2024A05 and Grant PCL2025A11, and Grant PCL2025A13, in part by the National Natural Science Foundation of China under Grant U24B20151, in part by Guangdong Major Project of Basic and Applied Basic Research under Grant 2019B030302002, and in part by China Postdoctoral Science Foundation under Grant 2025M771700. (Guoxiang Zhong and Jie Liu contributed equally to this work.) (Corresponding authors: Fagui Liu; Jun Jiang.)

Guoxiang Zhong, Jun Jiang, Bin Wang, and Fan Zhang are with the Department of New Networks, Pengcheng Laboratory, Shenzhen 518000, China (e-mail: zhonggx01@pcl.ac.cn; zhangf@pcl.ac.cn; junjiang@ieee.org; wangb02@pcl.ac.cn).

Jie Liu and C. L. Philip Chen are with the School of Computer Science and Engineering, South China University of Technology, Guangzhou 510006, China (e-mail: seliujie@scut.edu.cn; philip.chen@ieee.org).

Fagui Liu is with the School of Computer Science and Engineering, South China University of Technology, Guangzhou 510006, China, and also with the Department of New Networks, Pengcheng Laboratory, Shenzhen 518000, China (e-mail: fglu@scut.edu.cn).

Digital Object Identifier 10.1109/TCSS.2025.3649202

¹<https://openai.com/blog/chatgpt/>

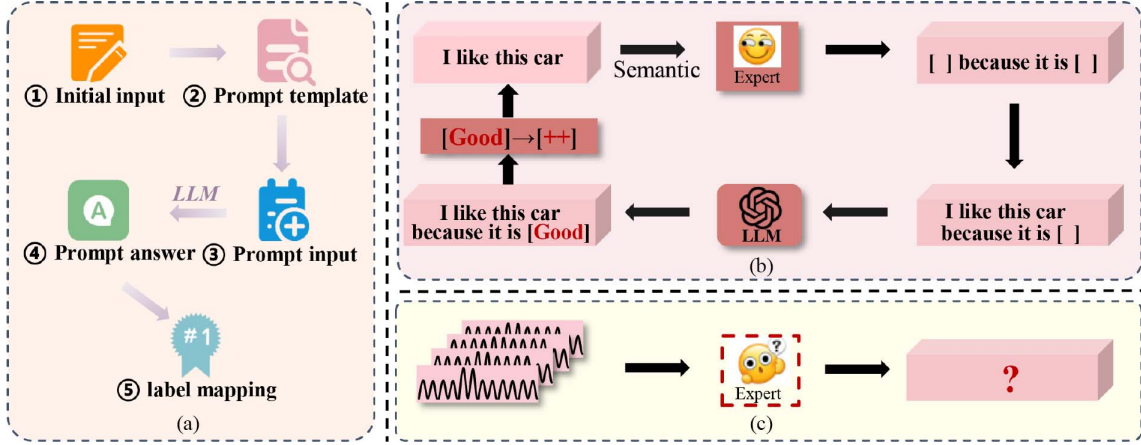


Fig. 1. Constructing the prompt: (a) for fine-tuning the text-based; (b) time-series-based; and (c) downstream tasks.

multivariate time series anomaly detection (LLM-AD). More specifically, the masked operation without unsupervised information is renovated to compose the prompt. We mask the variates rather than time stamp samples, which is referred to as the variate-level prompt. The unmasked variates act as the prefix component, which corresponds to the text-based prefix prompt template. The LLMs fill the masked variates or empty slots through fine-tuning. Notably, GPT-2, the popular variant of LLMs, is absorbed into our LLM-AD. Only the multihead self-attention block has been frozen. The activated feedforward and normalization layers lighten the training burden and retain acceptable detection accuracy. We annotate the testing time series according to the discrepancy between their answered and actual variates. Extensive experiments are conducted on four benchmark datasets. Our LLM-AD acquires an over 6% improvement of the averaged f1-score.

The main contributions of our LLM-AD algorithm are summarized as follows.

- 1) We introduce an LLM-based anomaly detector, which further accelerates the application of LLMs for more anomaly detection tasks.
- 2) We design the specialized prompt variety for multivariate time series anomaly detection, which facilitates the development of prompt learning theory architecture.
- 3) Our method achieves competitive or superior results when compared with the existing state-of-the-art baselines. The Python-based code of LLM-AD will be open-source at GitHub later.

II. RELATED WORK

Our LLM-AD is mainly concerned with two parts of background work: prompt learning for time series and time series anomaly detection. We systematically discuss them to speed up the understanding.

A. Prompt Learning for Time Series

The early prompt learning in the NLP area is one of the most robust tools to bring LLMs into the downstream tasks [24], [25]. Specifically, the raw textual input cooperates with

an extra prompt template to form the prompt input. The pre-trained LLMs yield the prompt answer which can be mapped as the task-specific label [23]. Subsequently, some CV-based tasks also explore prompt learning, which takes a similar operation as NLP and demonstrates its effectiveness. Being inspired by these empirical researches, the LLMs-driven time series tasks add the prior textual information into numerical sequence so as to construct the prompt. On the one hand, the manual discrete prompt purely wraps word sentences into time series. PromptCast [20] deploys the textual task background, which absorbs the past numerical time series as a natural language sentence for forecasting future time stamps. LSTPrompt [26] follows the *Chain-of-Thought* [27] prompt strategy and adds textual sequence to describe the characteristics of two decomposed time series forecasting subtasks. On the other hand, the automated prompt inclines towards the word embedding, rather than the explicit and bare word. TEST [28] optimizes the objective loss function between task actual and LLMs' predicted labels to formulate the task-specific embedding space, which aligns the time series token embedding and text embedding to construct the learnable prompt. TEMPO [21] introduces the prompt pool [29] which is equipped with the word embedding key-value pairs. The combination of input time series embedding and its top-k values from the prompt pool is fed into the LLMs. These two kinds of approaches are not contradicted but complementary to each other. The word in the discrete prompt emphasizes the prior expert knowledge, while the automated prompt requires the network training process to acquire the word embedding. Some published works present a better compromise between both of them. For example, Time-LLM [22] denotes the *Prompt-as-Prefix* encompassing patch reprogramming and dataset description. The former employs multihead attention to integrate time series patch embedding and text prototype, which come from pre-trained word embedding. The latter, with dataset context, task instruction, and input statistics, is another part of the input for the body pre-trained LLMs. GPT4MTS [30] utilizes the LLMs to collect the prior textual input, which concatenates the time series embedding after the BERT embedding layer [31].

Distinct from these prompt-based approaches, our LLM-AD actually proposes the textless prompt which does not carry any word information to LLMs.

B. Time Series Anomaly Detection

The anomaly detection is a critical but challenging task for time series analysis. The deep encoder-decoder-based algorithms using reconstruction theory have become mainstream in the past decade. In detail, the encoder transforms the initial multivariate time series into the lower-dimension latent space. Then, the decoder rebuilds the input, which is optimized with the training dataset encompassing most of the normal samples. The anomaly time stamp denotes the larger reconstruction error using this trained architecture. The capacity of formulating the spatial-temporal dependency is essential for this kind of method. GRELEN [12] employs the graph to consider the between-sensor dependence relationship. CAE-M [10] establishes a deep convolution auto-encoder for spatial dependence and combines auto-regressive and bidirectional LSTM for temporal dependence. Significantly, the Transformer-based [32] technique is one of the most powerful developments for focusing on context relationships between sequence-based data points. TranAD [13] further integrates adversarial training and model-agnostic meta learning into transformer. Anomaly transformer [14] upgrades the classic attention mechanism and proposes a new anomaly-attention block for capturing the association discrepancy. Both TranAD and anomaly transformer reach the comparable achievements about detection accuracy. Another line of research attempts to develop the diffusion-based detector due to their excellent performances in time series analysis [33], [34]. D³R [35] proposes noise diffusion to overcome the information bottleneck which is a common issue encountered in reconstruction-based methods. The spatial-temporal transformer backbone then denoises the polluted input data generated by this noise diffusion process. Besides, some general time series analysis algorithms also cover anomaly detection. For example, TimesNet [36] converts the 1-D time series into a 2-D tensor by discovering multiple periods. The parameter-efficient inception block forms the backbone network. ModernTCN [37] also adopts the convolution structure to describe cross-time and cross-variable dependency. TimesNet and ModernTCN benefit from improved convolution network block. In the meanwhile, some recent research efforts have concentrated on LLM-based anomaly detector. One-Fits-All [38] straightforwardly introduces the frozen LLMs for a unified framework about seven major time series downstream tasks, including anomaly detection. UniTS [39] develop sequence attention, variable attention, and dynamic MLP to improve basic Transformer block in LLM architecture. The unified pretraining consisting of regression and classification losses aims at a unified times series model.

Different from these models, our LLM-AD innovates the imputation theory to replace the reconstruction procedure. In other words, our LLM-AD only focuses on rebuilding a minority of variates by the pretrain LLM, instead of the whole time stamp sample.

III. METHODOLOGY

We first formulate the under-addressed problem and then develop the corresponding solution. The model structure with the proposed solution is revealed at last.

A. Problem Formulation

The multivariate time series $\mathcal{X}$ has been given. From time stamp 1 to t , this $\mathcal{X}$ can be organized as

$$\begin{aligned}\mathcal{X} &= \{x_i \mid i = 1, 2, \dots, t\}, \\ x_i &= [x_{i1}, x_{i2}, \dots, x_{id}] \in \mathbb{R}^d\end{aligned}\quad (1)$$

where $d > 1$ is the number of variates or dimensions. The x_i is the i th time stamp sample of $\mathcal{X}$. The anomaly detector $G(\cdot)$ predicts the label $\mathcal{Y}$ based on $\mathcal{X}$

$$\begin{aligned}\mathcal{Y} &= G(\mathcal{X}), \\ \mathcal{Y} &= \{y_i \mid i = 1, 2, \dots, t\}.\end{aligned}\quad (2)$$

The $y_i = 0$ and $y_i = 1$ individually represent the normal and anomaly labels for x_i . The $G(\mathcal{X})$ absorbs the pre-trained LLMs and then improves the downstream detection task through prompt learning. For the first step, the prompting function f_p converts the input $\mathcal{X}$ into a prompt input $\mathcal{X}^*$ with some additional textual information ρ

$$\mathcal{X}^* = f_p([\mathcal{X}, \rho]).\quad (3)$$

Then the LLMs fill the text-based prompt and searches for the highest-scoring prompt answer $\mathcal{Y}^*$

$$\mathcal{Y}^* = LLM(\mathcal{X}^*).\quad (4)$$

At last, the answer mapping function f_l is applied to modify $\mathcal{Y}^*$ into the final task-specific label $\mathcal{Y}$

$$\mathcal{Y} = f_l(\mathcal{Y}^*).\quad (5)$$

Thus, the $G(\mathcal{X})$ also can be denoted as

$$G(\mathcal{X}) \equiv f_l \circ LLM \circ f_p([\mathcal{X}, \rho]).\quad (6)$$

(Problem) The prompting function f_p aligns numerical time series $\mathcal{X}$ with textual information ρ (word string or embedding) for the prompt input $\mathcal{X}^*$ of LLMs. However, it is difficult for f_p to acquire the ρ of time series. In the meanwhile, multivariate time series anomaly detection always occurs in an unsupervised manner. For solving this problem, we design an unsupervised textless prompting function f_p without ρ

$$\mathcal{X}^* = f_p(\mathcal{X}).\quad (7)$$

B. Prompt Modeling Based on Masked Operation

We draw a random Boolean masked matrix B with a masked ratio α ($0 < \alpha < 1$), and acquire two complementary and masked parts of $\mathcal{X}$

$$\begin{aligned}\mathcal{X}^{(1)} &= \mathcal{X} \odot B, \\ \mathcal{X}^{(2)} &= \mathcal{X} \odot (1 - B)\end{aligned}\quad (8)$$

where $\odot$ represents the Hadamard product. The $\mathcal{X}^{(1)}$ and $\mathcal{X}^{(2)}$ also are $t \times d$ matrices satisfying

$$\begin{aligned} \|\mathcal{X}^{(1)}\|_0 &= \|B\|_0 = \lceil (1 - \alpha)td \rceil, \\ \|\mathcal{X}^{(2)}\|_0 &= \lfloor \alpha td \rfloor, \\ \|\mathcal{X}^{(1)}\|_0 + \|\mathcal{X}^{(2)}\|_0 &= td. \end{aligned} \quad (9)$$

We further formulate this random masked operation as

$$\{\mathcal{X}^{(1)}, \mathcal{X}^{(2)}\} = f_m(\mathcal{X}). \quad (10)$$

The first output value $\mathcal{X}^{(1)}$ is referred to as prompt input for the LLMs, while the second one $\mathcal{X}^{(2)}$ is prompt answer. Our textless prompt approach removes the text-related prompt template which bridges the initial input between prompt answer in NLP tasks. For being consistent with the input of the pre-trained LLMs, we employ the zero-padding operation with fully connected neural network layer. Our masked-operation-based prompt takes a form as

$$\mathcal{X}^* = PAD(\mathcal{X}^{(1)}). \quad (11)$$

In other words, $f_p(\cdot) = f_m \circ PAD(\cdot)$. The LLMs maps $\mathcal{X}^*$ into $\hat{\mathcal{X}}^{(2)}$, which acts as the procedure of filling the empty prompt slots about variates

$$\hat{\mathcal{X}}^{(2)} = LLM(\mathcal{X}^*). \quad (12)$$

The predicted prompt answer $\hat{\mathcal{X}}^{(2)}$ is optimized with the MSE loss function during fine-tuning the LLMs

$$\mathcal{L}(\mathcal{X}^{(2)}, \hat{\mathcal{X}}^{(2)}) = \|\mathcal{X}^{(2)} - \hat{\mathcal{X}}^{(2)}\|_2^2. \quad (13)$$

The reason why this prompt model reaches the desirable performance is that the fine-tuned LLMs with the pattern of normal behavior cannot rebuild the masked variates according to anomaly variates. Anomalies in multivariate time series often manifest as violations of the learned dependencies or structures. When an anomalous timestamp is encountered during testing, a subset of its variables is masked. The unmasked part may provide contradictory or inconsistent signals compared with the what the model learned during training on normal data. The inherent dependencies needed to accurately reconstruct the masked variables based on this anomalous context are weakened, which leads to terrible variates reconstruction. The reconstruction error for the masked variables at this timestamp becomes significantly higher compared with the errors at normal timestamp. Without loss of generality, we annotate $x_i \in \mathcal{X}$ as $y_i = 1$. The first m variates $x_{i,1:m} = [x_{i1}, x_{i2}, \dots, x_{im}]$ includes root-cause ones, which constructs the prompt

$$\begin{aligned} \hat{x}_{i,m+1:d} &= LLM(PAD(x_{i,1:m})), \\ x_{i,m+1:d} &\approx \hat{x}_{i,m+1:d} \end{aligned} \quad (14)$$

where the output $\hat{x}_{i,m+1:d}$ is far away from the actual $x_{i,m+1:d}$.

As shown in Fig. 2, several variates lead to the anomaly behavior while the others still be unaffected. We suppose that each anomaly time stamp has d_r root-cause variates. Our prompt

TABLE I
ROOT CAUSE VARIATES OF ANOMALY TIME STAMP

Anomaly Time Stamp	Root Cause Variate
15849-16368 ¹	1, 9, 10, 12, 13, 14, 15 ²
16963-17517	1, 2, 3, 4, 6, 7, 9, 10, 11, 12, 13, 14, 15, 16, 19, 20, 21, 22, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36
18071-18528	1, 2, 9, 10, 12, 13, 14, 15
19367-20088	1, 2, 3, 4, 9, 10, 11, 12, 13, 14, 15, 16, 25, 28
20786-21195	1, 9, 10, 12, 13, 14, 15
24679-24682	9, 13, 14, 15
26114-26116	9, 13, 14, 15
27554-27556	9, 13, 14, 15

Note: ¹The #th time stamp of the time series.

²The #th variate of the time series.

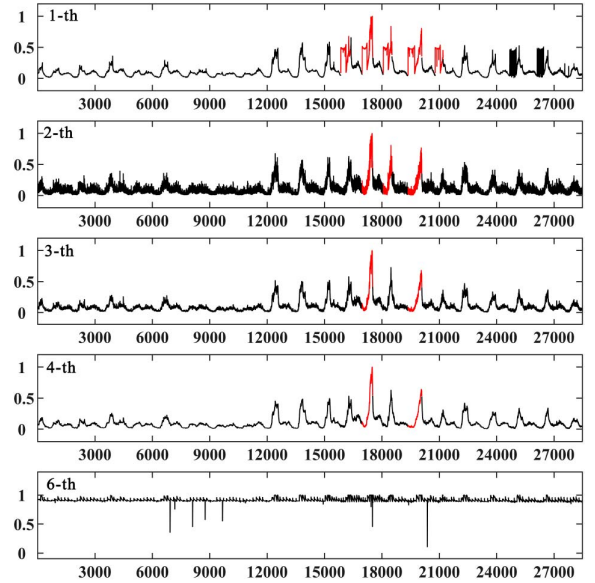


Fig. 2. 1th, 2th, 3th, 4th, and 6th variates about the first testing machine of SMD [5]. We do not plot the curve of the 5th variate because it is kept to zero all the way. The other root cause variates are in Table I. The curve in red indicates the anomaly time stamp.

model masks d_m variates. The probability that at least one root-cause variates are unmasked is shown as follows:

$$p = 1 - \frac{d - d_r}{d} \times \frac{d - d_r - 1}{d - 1} \times \dots \times \frac{d - d_r - (d - d_m - 1)}{d - (d - d_m) + 1}. \quad (15)$$

If $d_r < d_m = \lceil (1 - \alpha)d \rceil$, then $p = 1$. For the example in Table I and Fig. 2, $d_r \geq 4$, our prompt need to select more than 34 unmasked variates, which ensure the efficient detection.

C. Model Structure

The LLM-AD architecture is depicted in Fig. 3. For the time series analysis task, the contextual features between different time stamps are critical. Consequently, the input $\mathcal{X}$ of the related algorithms is translated as slide windows

$$\mathcal{X} = \{W_j \mid j = 1, 2, \dots, s\},$$

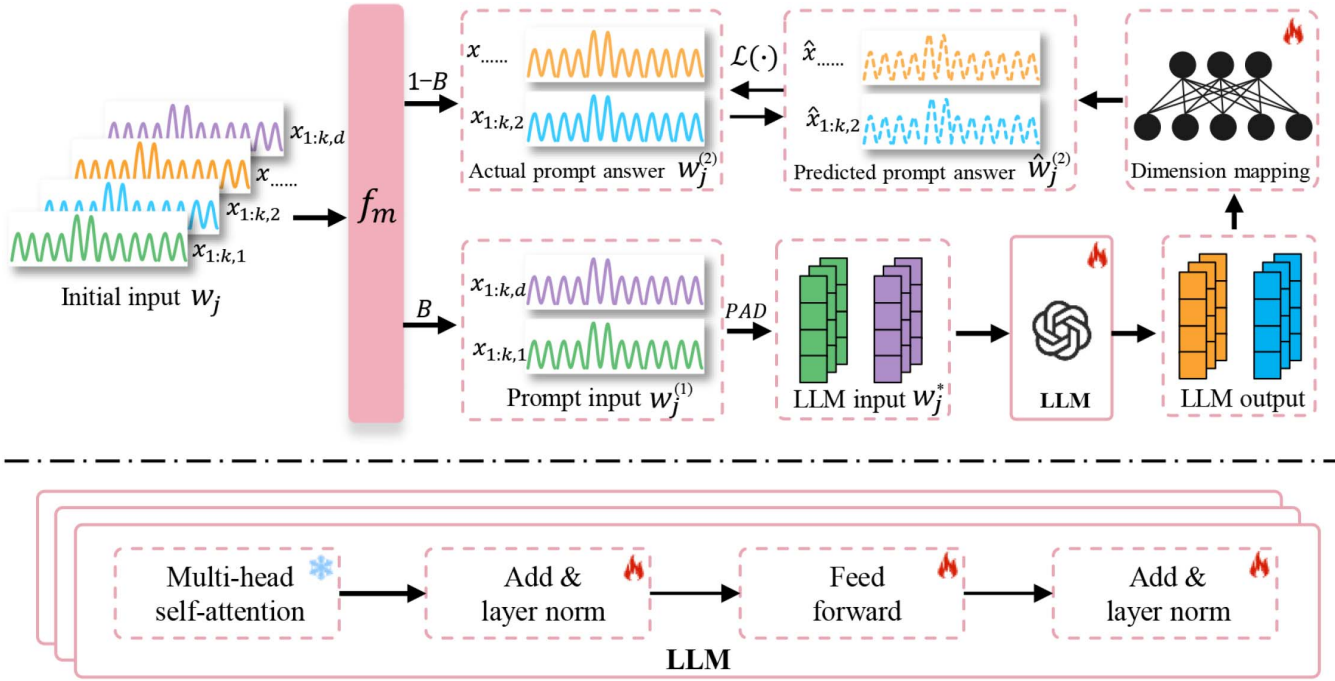


Fig. 3. Model framework of LLM-AD. The input time series window W_j is divided into the input component $W_j^{(1)}$ and answer component $W_j^{(2)}$ of the prompt by masked operation f_m . We fine-tune the LLMs and generate the predicted answer $\hat{W}_j^{(2)}$. The MSE loss function $\mathcal{L}(\cdot)$ between $\hat{W}_j^{(2)}$ and $W_j^{(2)}$ optimize the training procedure. The margin between predicted prompt answer and actual prompt answer decides the predicted label of testing time series. The $\rightarrow$ indicates the dataflow direction of LLM-AD training and testing procedures. The $\rightleftarrows$ presents the loss function computation about training algorithm.

$$W_j = [x_{1:k,1}, x_{1:k,2}, \dots, x_{1:k,d}]_j \quad (16)$$

where k is window size and $s = \lfloor t/k \rfloor$ is the number of windows. For the j th window W_j in $\mathcal{X}$, the masked operation converts it into $W_j^{(1)}$ and $W_j^{(2)}$. The former creates the prompt input which is fed into the LLMs for searching the predicted answer. Significantly, the pre-trained parameters of the multihead self-attention block in LLMs are retained. We only retrain the normalization layers and feedforward layers. The pseudo code in Algorithm 1 presents the implementation.

During the testing stage, the testing time series $\tilde{\mathcal{X}}$ is given. Our goal is to assign $\tilde{\mathcal{X}}$ with its predicted label $\tilde{\mathcal{Y}}$

$$\begin{aligned} \tilde{\mathcal{X}} &= \{\tilde{W}_j \mid j = 1, 2, \dots, l\}, \\ \tilde{W}_j &= [\tilde{x}_{1:k,1}, \tilde{x}_{1:k,2}, \dots, \tilde{x}_{1:k,d}]_j, \\ 1 &\leq j \leq l \end{aligned} \quad (17)$$

where l is the number of windows. For the j th testing window $\tilde{W}_j$, the finished $LLM \circ PAD \circ f_m(\cdot)$ offer an answer $\tilde{W}_j^{(2)}$ according to $\tilde{W}_j^{(1)}$

$$\tilde{W}_j^{(2)} = LLM \circ PAD \circ f_m(\tilde{W}_j^{(1)}). \quad (18)$$

The divergence between actual $\tilde{W}_j^{(2)}$ and predicted $\tilde{W}_j^{(2)}$ denotes the criterion of labeling time stamp. We also consider the following criterion as classifier $f_t(\cdot)$ in $G(\cdot)$

$$\tilde{Y}_j = \begin{cases} 0, & \|\tilde{W}_j^{(2)} - \tilde{W}_j^{(2)}\|_2^2 \leq \theta \\ 1, & \|\tilde{W}_j^{(2)} - \tilde{W}_j^{(2)}\|_2^2 > \theta \end{cases} \quad (19)$$

Algorithm 1: LLM-AD Training Algorithm.

Require: Training time series $\mathcal{X}$; Boolean masked matrix B ; Pre-trained $LLM(\cdot)$; Iteration limitation N_e .

Ensure: Optimized $LLM(\cdot)$.

- 1: Divide $\mathcal{X}$ into window set $\{W_1, W_2, \dots, W_s\}$;
- 2: $e \leftarrow 1$;
- 3: **repeat**
- 4: $j \leftarrow 1$;
- 5: **repeat**
- 6: Confirm the prompt input $W_j^{(1)} = W_j \odot B$;
- 7: Pad the LLMs input $W_j^* = PAD(W_j^{(1)})$;
- 8: Construct the prompt answer $W_j^{(2)} = W_j \odot (1-B)$;
- 9: Feed W_j^* into $LLM(\cdot)$;
- 10: Map the LLMs' output as $\hat{W}_j^{(2)}$;
- 11: Optimize $LLM(\cdot)$ by MSE loss function;
- 12: $j \leftarrow j+1$;
- 13: **until** ($s = j$).
- 14: $e \leftarrow e+1$;
- 15: **until** ($N_e = e$).

where $\tilde{Y}_j \in \tilde{\mathcal{Y}}$ is the final label of $\tilde{W}_j$. Last but not least, the above-mentioned criterion requires confirming the threshold θ . More proposed works [40], [41] have proposed the relevant approaches. The pseudo code description about testing is shown in Algorithm 2.

Algorithm 2: LLM-AD Testing Algorithm.

Require: Optimized $LLM(\cdot)$; Testing time series $\tilde{\mathcal{X}}$; Boolean masked matrix B ; Classifier threshold θ .

Ensure: Label $\tilde{\mathcal{Y}}$ of $\tilde{\mathcal{X}}$.

```

1: Split  $\tilde{\mathcal{X}}$  into window sequence  $\{\tilde{W}_1, \tilde{W}_2, \dots, \tilde{W}_l\}$ ;
2:  $\mathcal{Y} \leftarrow \emptyset$ ;
3:  $j \leftarrow 1$ ;
4: repeat
5:   Generate  $\tilde{W}_j^{(1)}$  and  $\tilde{W}_j^{(2)}$  using  $B$ 
6:   Upload the  $PAD(\tilde{W}_j^{(1)})$  into  $LLM(\cdot)$ ;
7:   Compute the divergence between  $\tilde{W}_j^{(2)}$  and  $\tilde{W}_j^{(1)}$ ;
8:   Confirm  $\tilde{Y}_j$  using threshold  $\theta$ ;
9:    $\mathcal{Y} = \{\mathcal{Y}; \tilde{Y}_j\}$ ;
10:   $j \leftarrow j + 1$ ;
11: until ( $l = j$ ).
```

TABLE II
CHARACTERISTICS OF BENCHMARK DATASETS

Dataset	SMD	MSL	SMAP	SWAT
Training	708 405	58 317	135 183	495 000
Testing	708 420	73 729	427 617	449 919
Dimension	38	55	25	51
Anomaly ratio	4.16%	10.72%	12.79%	12.14%

IV. EXPERIMENT

In this section, we validate our proposed LLM-AD by answering the following five research questions (*RQs*), which belong to the comparison experiment, ablation experiment, and parameter sensitivity analysis.

- RQ1*: Does LLM-AD get competitive detection results?
- RQ2*: How does the proposed component affect LLM-AD?
- RQ3*: How does backbone LLM influence our LLM-AD?
- RQ4*: How does unmasked ratio affect our LLM-AD?
- RQ5*: How does window contribute to our LLM-AD?

A. Dataset Details

All of the experiments are conducted on four datasets, including SMD [5], MSL [6], SMAP [6], and SWAT [7]. The first one is from the computer servers cluster, the second two are collected from space exploration, and the last one comes from secure water treatment in the modern industrial control system. Our LLM-AD can be substantiated using various datasets with different sizes, different dimensions, and different anomaly ratios. More information about these datasets is in Table II.

B. Evaluation Metrics

In our experiments, we apply recall (R), precision (P), and f1-score ($F1$) to evaluate the performance of anomaly detectors, including our LLM-AD and other comparative baselines

$$R = \frac{TP}{TP + FN}, P = \frac{TP}{TP + FP}, F1 = \frac{2 \cdot P \cdot R}{P + R} \quad (20)$$

where TP , FN , and FP are the true positive, false negative, and false positive about anomaly sample, respectively.

C. Model Configuration

The unmasked ratio in our proposed prompt model is 0.95. This prompt is fed into the GPT-2 backbone. We experiment with GPT-2 Medium [42], the 355M parameter version of GPT-2, and download it from this URL². For dimension alignment of the output time series, The single fully connected neural network layer ($1024 \rightarrow d$) with layer normalization is added following the GPT-2 Medium. Our experimental platform is equipped with 8 NVIDIA GeForce RTX 2080Ti GPUs. We deploy the algorithm-used Python and other depended-upon packages, such as PyTorch³ and Transformer⁴, into this experimental computation server. The training of our LLM-AD is optimized by the Adam optimizer [43] with a learning rate of 10^{-5} . To avoid over-fitting, we implement gradient clipping while the l_2 -norm of all gradients reaches 200. The batch size is 64, while the number of epochs is 50. We use the no-overlapping window with a size of 100. During real-world practice, only one of the presented anomaly time stamps is correctly identified while the remaining ones in the same windows would arouse the vigilance. Thus, more published works adopt the point-adjustment evolution strategy [13], [14], [44]. We also take advantage of the identical operation. Threshold selection in the final classifier is necessary for our LLM-AD and other baselines. We confirm the threshold through the precision-recall curve, where the classification threshold proportions are in $\{0.01\%, 0.02\%, 0.05\%, 0.1\%, 0.2\%, 0.5\%, 1.0\%, 2.0\%, 5.0\%, 10\%, 20\%, 30\%\}$. Fig. 4 displays the precision-recall curve of LLM-AD.

D. Results and Analysis

The experiment results and their analysis of the corresponding five *RQs* are displayed. The superiority of our LLM-AD algorithm has been highlighted.

1) *RQ1: Does LLM-AD Get Competitive Detection Results?*: This *RQ* is involved with the competition between LLM-AD and other baseline algorithms, containing FEDformer [45], DLinear [46], TimesNet [36], One-Fits-All [38], GRELEN [12], TranAD [13], Anomaly Transformer [14], CAE-M [10], AnomalyLLM [47], aLLM4TS [48], and AnomalyBERT [49]. The FEDformer and DLinear aim at the time series forecasting. However, they are also referred to as anomaly detection baselines in the validation experiment of general time series analysis methods [36], [38]. The TimesNet and One-Fits-All focus on all of the common time series tasks. The GRELEN, TranAD, Anomaly Transformer, CAE-M, and AnomalyBERT are dedicated to time series anomaly detection. In particular, both TranAD and Anomaly Transformer evolve from Transformer. AnomalyLLM employs a knowledge distillation-based framework where the training optimization goal is to minimize the distance between the outputs of transformer-based

²<https://huggingface.co/openai-community/gpt2-medium>

³<https://pytorch.org/>

⁴<https://github.com/huggingface/transformers>

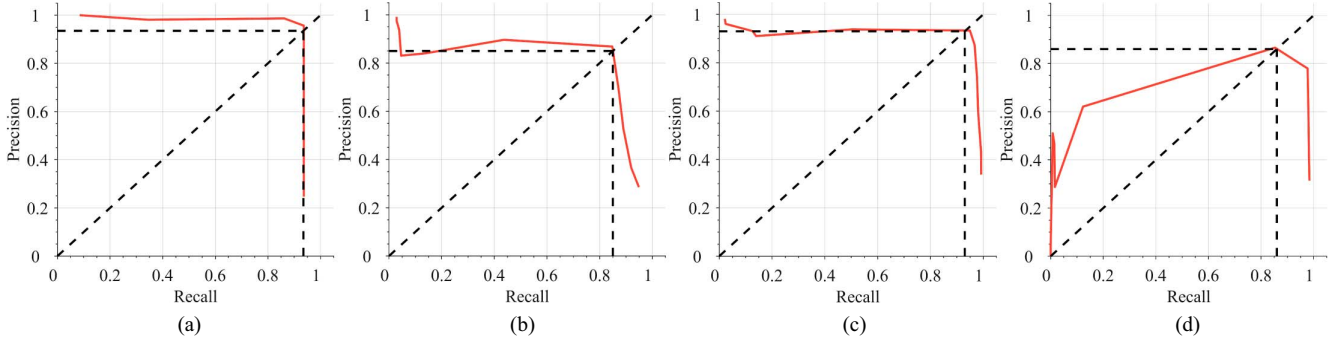


Fig. 4. Precision-recall curve of LLM-AD in four datasets. For MSL and SMAP, the reason for local smaller precision at the beginning is that part of actual normal samples are regarded as anomalies. For SWAT, the anomaly threshold is too large, which leads to fewer anomaly sample being identified. Thus, both the precision and recall are simultaneously close to zero. (a) SMD. (b) MSL. (c) SMAP. (d) SWAT.

TABLE III
DETECTION RESULTS OF OUR LLM-AD AND NINE BASELINES IN FOUR BENCHMARK DATASETS

Algorithm	$\bar{F1}^1$	$\bar{R}$	$\bar{P}$	SMD			MSL			SMAP			SWAT		
				F1	R	P	F1	R	P	F1	R	P	F1	R	P
FEDformer	76.77	69.45	88.39	76.95	81.57	72.79	82.25	75.25	<u>90.65</u>	68.67	55.42	90.18	79.21	65.57	99.95
DLinear	80.49	76.65	86.99	80.07	84.46	76.12	81.89	75.30	89.69	67.31	53.79	89.87	<u>92.68</u>	93.05	92.27
TimesNet	82.14	76.62	89.91 ²	84.62	81.54	87.89	81.82	75.29	89.55	69.51	56.55	90.09	92.62	93.09	92.11
One-Fits-All	81.73	77.69	87.83	83.98	80.79	87.43	81.41	81.09	81.69	68.90	55.80	90.07	92.61	93.08	92.10
GRELEN	72.54	73.07	79.80	89.39	94.42	84.86	<u>88.09</u>	<u>91.66</u>	84.78	34.57	22.36	76.16	78.13	83.85	73.39
TranAD	67.45	59.71	83.93	92.61	<u>99.87</u>	86.25	60.88	47.87	83.60	44.06	31.23	74.77	72.26	59.88	91.09
Anomaly Transformer	84.33	86.46	82.98	82.44	81.60	83.30	75.18	84.09	67.97	93.26	99.03	88.12	86.45	81.12	92.52
CAE-M	80.17	80.19	82.85	82.84	76.65	90.12	82.38	82.32	82.43	82.38	99.69	70.04	73.09	62.09	88.82
AnomalyLLM	72.80	65.99	86.30	93.21	95.03	91.46	81.99	80.09	83.97	42.10	30.04	70.36	73.91	58.82	99.40
aLLM4TS	82.66	79.91	85.90	83.98	80.91	87.26	80.18	79.31	81.02	75.27	67.91	84.40	91.22	91.52	90.92
AnomalyBERT	85.28	83.61	87.46	83.49	80.94	86.21	75.09	69.87	81.14	91.29	96.02	87.26	91.27	87.63	95.23
LLM-AD	<u>91.62</u> ²	<u>92.64</u>	<u>90.74</u>	<u>94.64</u>	93.65	<u>95.64</u>	85.78	84.79	86.79	<u>94.12</u>	94.93	<u>93.32</u>	91.93	<u>97.19</u>	87.22

Note: ¹The averaged results of four datasets are over-lined.

²The second best result is highlighted in **bold**, and the best performance is underlined.

student network and GPT-based teacher network. This distance is also referred to as the anomaly score during the testing stage. aLLM4TS proposes a self-supervised multipatch prediction approach for general time series representation learning using an LLM as the backbone network. AnomalyBERT unleashes the ability of BERT backbone [31], which is also one of the most powerful pre-trained language models.

Table III presents the experiment results about the *RQ1*. In addition to the initial recall, precision, and f1-score, their averaged values of four datasets are also displayed. Our LLM-AD gets the highest $\bar{R}$ (92.64%) and $\bar{P}$ (90.74%), outperforming the state-of-the-art baseline with an increase of 6.34% (85.28% $\rightarrow$ 91.62%) in $\bar{F1}$. We ascribe this result to the best f1-score and precision in SMD and SMAP. In the meanwhile, the second best f1-score (85.78%) and recall (84.79%) in MSL and the best recall (97.19%) also achieve their contributions. The GPT-2 strengthens the backbone generalization ability for time stamp dependency. More presented anomaly samples are precisely detected, which is another reason (see confusion matrix in Fig. 5).

For the comparative algorithm, AnomalyBERT has the second highest $\bar{F1}$ (85.28%), which also relies on the strong

backbone network. The dedicated anomaly-attention in the Anomaly Transformer leads to the second best $\bar{R}$ (86.46%). It is noteworthy that the general time series analysis algorithm TimesNet has the impressive performance of $\bar{P}$ (89.91%). Specifically, GRELEN surpasses our LLM-AD by the margins of 3.69% (85.78% $\rightarrow$ 88.09%) and 6.87% (84.79% $\rightarrow$ 91.66%) about the f1-score and recall in MSL. However, TranAD yields the best recall (99.87%) in SMD, and CAE-M reaches the best recall (99.69%) in SMAP. In conclusion, it is difficult to get the SOTA achievement for any anomaly detection algorithm in all benchmark datasets.

2) *RQ2: How Does the Proposed Component Affect LLM-AD?*: To answer this *RQ*, we take into account the proposed prompt model. Fig. 6 displays the comparison results. “With” is our initial LLM-AD. “Without” denotes the LLM-AD without textless prompt. Although the “Without” attains better recall in SMD and MSL, and better precision in SWAT, our “With” is prone to the higher f1-score. The reason is that fewer normal time stamps are identified as anomalies.

3) *RQ3: How Does Backbone LLM Influence Our LLM-AD?*: We first consider different backbone LLMs.

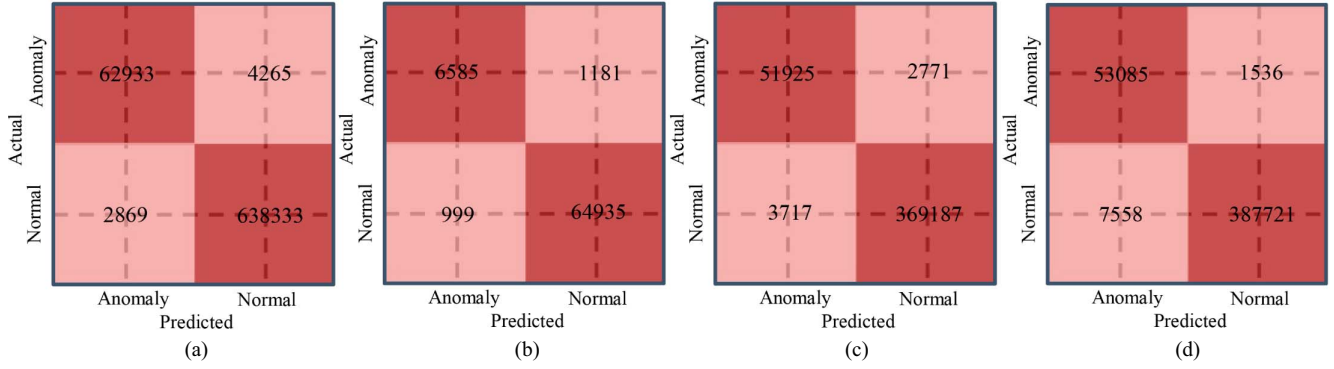


Fig. 5. Confusion matrices about four datasets. There is an extreme imbalance between the amounts of anomaly and normal samples for anomaly detection. For the confusion matrix, the anomaly sample belongs to the positive class, while the normal samples are in the negative class. In SMD and MSL, the number of false negatives is larger than false positives, which results in a larger precision. The contrary situation exists in SMAP and SWAT. Our LLM-AD attain the larger recall, compared with precision. (a) SMD. (b) MSL. (c) SMAP. (d) SWAT.

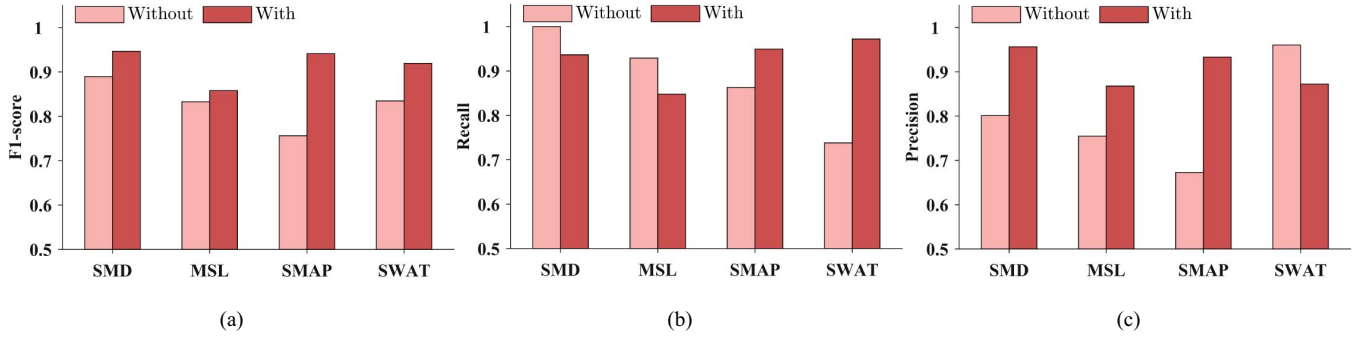


Fig. 6. Impact of the proposed prompt. (a) F1-score. (b) Recall. (c) Precision.

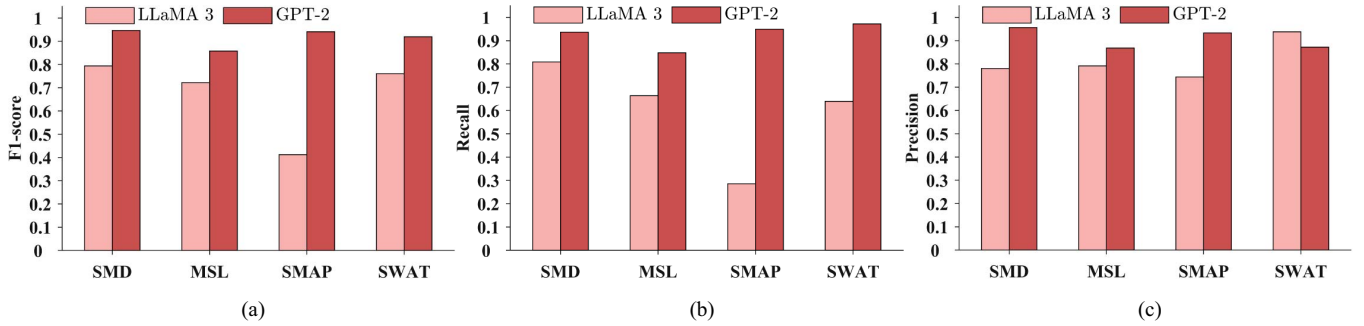


Fig. 7. Performance comparison of GPT-2 with LLaMA 3 backbone on LLM-AD. (a) F1-score. (b) Recall. (c) Precision.

The experiment results are displayed in Fig. 7. We introduce LLaMA 3 to replace GPT-2 in our LLM-AD. “LLaMA 3” and “GPT-2” indicate that LLaMA 3 and GPT-2 are the backbones of LLM-AD, respectively. Regarding f1-score and recall, “GPT-2” outperforms “LLaMA 3” by a large margin across four datasets. It is relatively easier for GPT-2 to understand the numerical sequence. For precision, “LLaMA 3” surpasses “GPT-2” in the SWAT dataset, which should be attributed to the fact that reducing false positives leads to an increase in false negatives.

Fig. 8 presents the effect of freezing various GPT-2 components. “MLP” and “Attn”, respectively indicate that only feedforward and multihead attention are frozen. Inversely, “MLP & Attn” means that we activate both feedforward and multihead attention layers. The “Attn” and “MLP & Attn” outperform the “MLP”. In some situations, the result margins between “Attn” and “MLP & Attn” are small. However, for SWAT, the “Attn” still surpasses “MLP & Attn” about three metrics. This phenomenon demonstrates that the feedforward layer, not the attention mechanism, plays the core role.

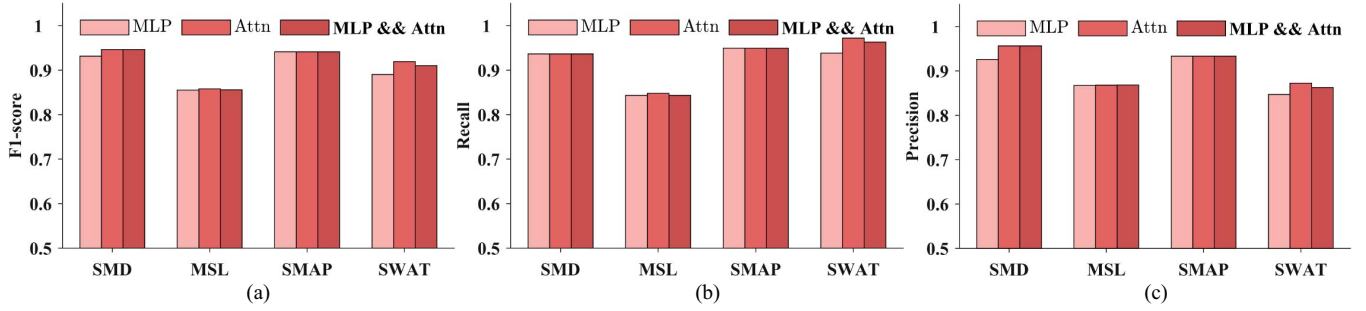


Fig. 8. Impact of different frozen GPT-2 network blocks. (a) F1-score. (b) Recall. (c) Precision.

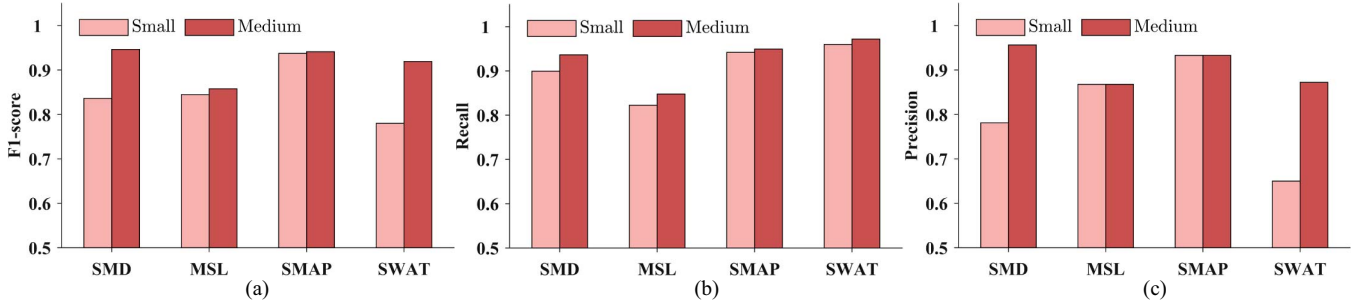


Fig. 9. Impact of different parameter size about GPT-2. (a) F1-score. (b) Recall. (c) Precision.

Furthermore, we consider the parameter size of GPT-2, which is related to network generalization ability. Fig. 9 pictures the comparison between GPT-2 Small (“Small”) and GPT-2 Medium (“Medium”). This URL⁵ provides the GPT-2 Small, which is the smallest version of GPT-2, with 124M parameters. The “Medium” exceeds the “Small” on the total metrics for four datasets. More Transformer encoders for the backbone network cause better detection accuracy (f1-score and precision) in SMD and SWAT.

Different frozen GPT-2 blocks and GPT-2 parameter sizes also impact the execution time of LLM-AD. The experiment results are shown in Tables IV and V. The time complexity of the feedforward layer and multihead self-attention mechanism are respectively $O(n)$ and $O(n^2)$, where n is the number of input samples. Although different frozen network blocks denote different scales of training network weights, their execution time in Table IV are still close to each other. The parameter size of GPT-2 Medium is approximately three times larger than GPT-2 Small, which provides the advantage of network generalization ability. In the meanwhile, Table V also indicates that the GPT-2 Medium has an execution time triple the GPT-2 Small.

4) *RQ4: How Does the Unmasked Ratio Affect Our LLM-AD?*: This *RQ4* is concerned with the effectiveness of our prompt model. Fig. 10 displays the experiment results. The unmasked ratio is up to 0.95, starting at 0.1 with an interval of 0.05. In the early stage, the inferior performance with a lower unmasked ratio (below 0.3) exists on all of the datasets. With the increase of the unmasked ratio, we acquire higher f1-score,

TABLE IV
EXECUTION TIME OF DIFFERENT FROZEN BLOCKS IN FOUR DATASETS

	SMD	MSL	SMAP	SWAT
MLP	6338.86 s	449.72 s	997.96 s	3559.38 s
Attn	6463.64 s	440.74 s	969.41 s	3432.97 s
MLP & Attn	6397.66 s	452.67 s	1020.85 s	3580.83 s

TABLE V
EXECUTION TIME OF DIFFERENT PARAMETER SIZES IN FOUR DATASETS

	SMD	MSL	SMAP	SWAT
Medium	6338.86 s	449.72 s	997.96 s	3559.38 s
Small	2200.85 s	156.44 s	350.73 s	1231.61 s

recall and precision, although fluctuation is also observed. The reason is that more variates are indispensable for describing the initial semantic feature of input time series. The classifier only requests a few variates for the indicator. While the unmasked ratio is equal to 0.95, most of the metrics reach the maximum values. Thus, we recommend the masked ratio be set as 0.95.

Our prompt answer is built upon the masked variates. We experiment with different numbers of masked variates according to different dataset scales. Table VI presents the masked variates using the random masked method with different random seeds. Fig. 11 shows the detection results with different masked variates in Table VI. For recall in SWAT and precision in SMAP,

⁵<https://huggingface.co/gpt2>

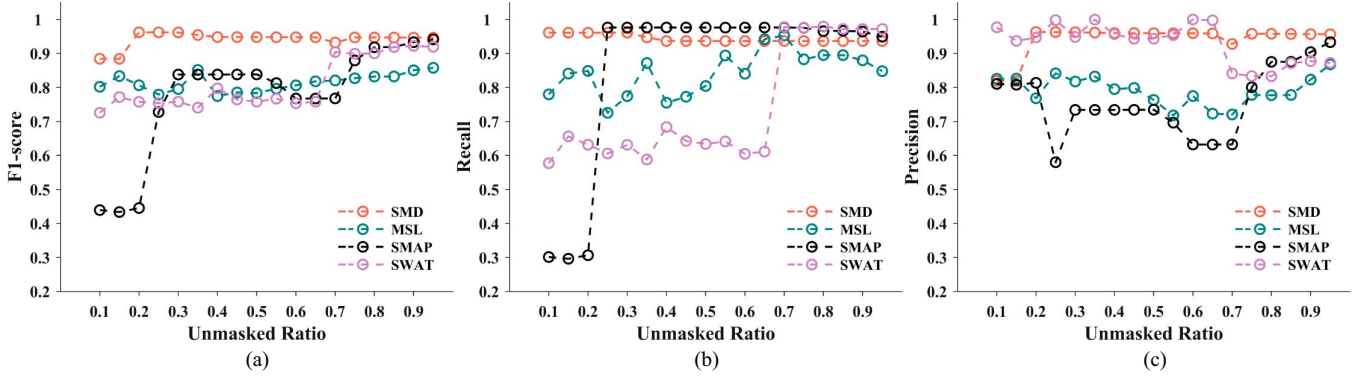


Fig. 10. Impact of unmasked ratio. (a) F1-score. (b) Recall. (c) Precision.

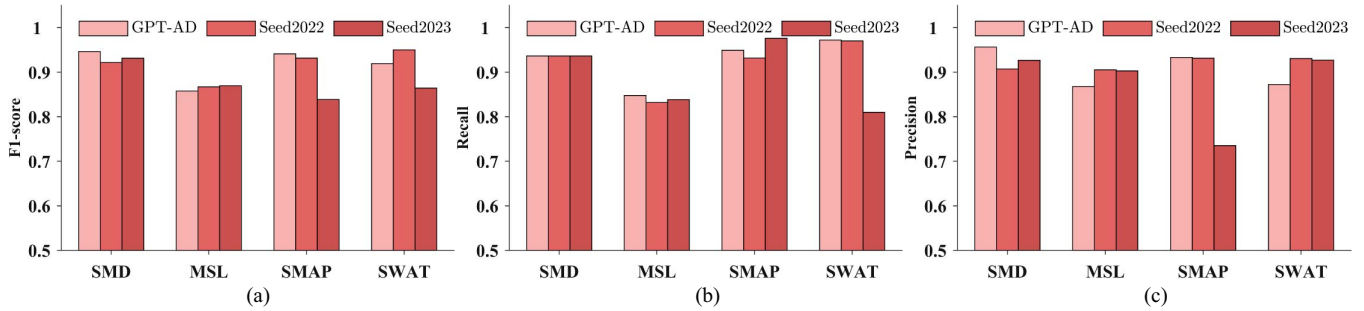


Fig. 11. Detection results using different masked variates. (a) F1-score. (b) Recall. (c) Precision.

TABLE VI
MASKED VARIATES IN FOUR DATASETS

Method	Masked Variates			
	SMD (2) ¹	MSL (3)	SMAP (2)	SWAT(3)
LLM-AD	37, 38	53, 54, 55	24, 25	49, 50 51
Seed=2022	25, 29	29, 46, 49	14, 17	29, 46, 49
Seed=2023	24, 26	7, 24, 26	7, 24	7, 24, 26

Note: ¹Number of masked variates.TABLE VII
EXECUTION TIME OF DIFFERENT WINDOW SIZES

Window Size	SMD	MSL	SMAP	SWAT
50	9545.23 s	714.41 s	1623.01 s	5876.26 s
100	6338.86 s	449.72 s	997.96 s	3559.38 s
130	5550.95 s	360.82 s	870.24 s	3038.52 s

the margin between “Seed = 2022” and “Seed = 2023” is still large.

5) RQ5: How Does Window Contribute to Our LLM-AD?:

We discuss the window size which affects the modeling of the temporal dependency relationship between time stamps. While the window size is too small, the long-term features will be ignored, and the detection time is also too long (see Table VII). On the contrary, the short-term properties can not

attract sufficient attention if the window size is large enough. At the same time, more memory and CPU resources are occupied in the computing platform. As shown in Fig. 12, the beginning window size is set to 30. The values of recall in four datasets are relatively small, while the precision is higher, which causes the stable f1-score. This is due to the best threshold about imputation error (see Fig. 13). For the point-adjustment evaluation, we require only a few anomalies that are truly discovered in a window. The window size comes to 100 with an interval of 10. The f1-score in SMD, MSL, and SMAP begin to drop dramatically, which ascribes to the serious decrease of precision. We suggest that the window size should be fixed between 50 and 100.

Furthermore, we add the discussion about nonoverlapping and overlapping windows. Fig. 14 displays the comparison results. “Win-150-100” denotes an overlapping window with size 150 and stride size 100. “Win-100-50” corresponds to an overlapping window of size 100 and sliding step 50. “Win-100” represents the nonoverlapping window which our LLM-AD utilizes. In most cases, “Win-100” outperforms “Win-150-100” and “Win-100-50”, which substantiates the superiority of nonoverlapping window in our LLM-AD. However, in terms of precision, “Win-100” falls behind “Win-150-100” and “Win-100-50” on the SWAT dataset. This is attributed to the fewer false negatives given by LLM-AD. The best F1-score and recall indicate that the nonoverlapping window is beneficial to the feature robust representation learning.

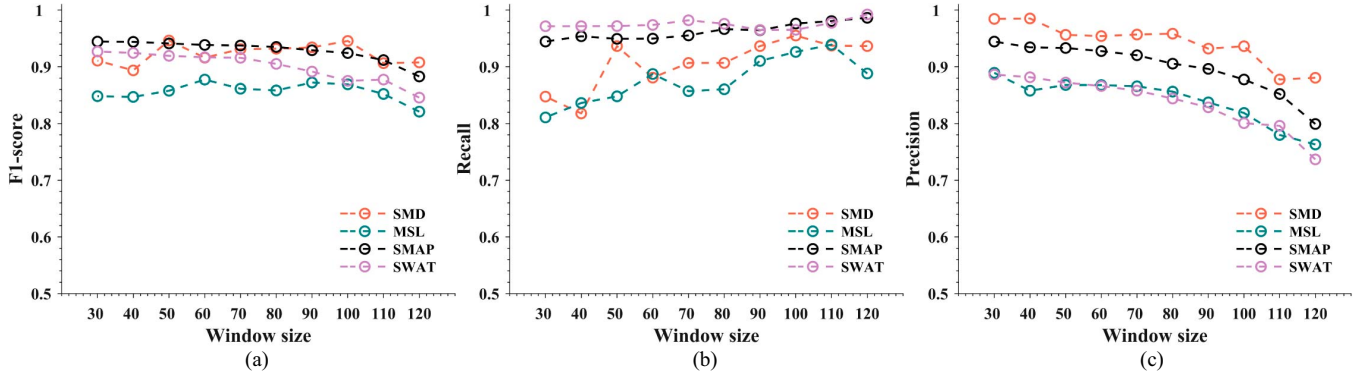


Fig. 12. Impact of window size. (a) F1-score. (b) Recall. (c) Precision.

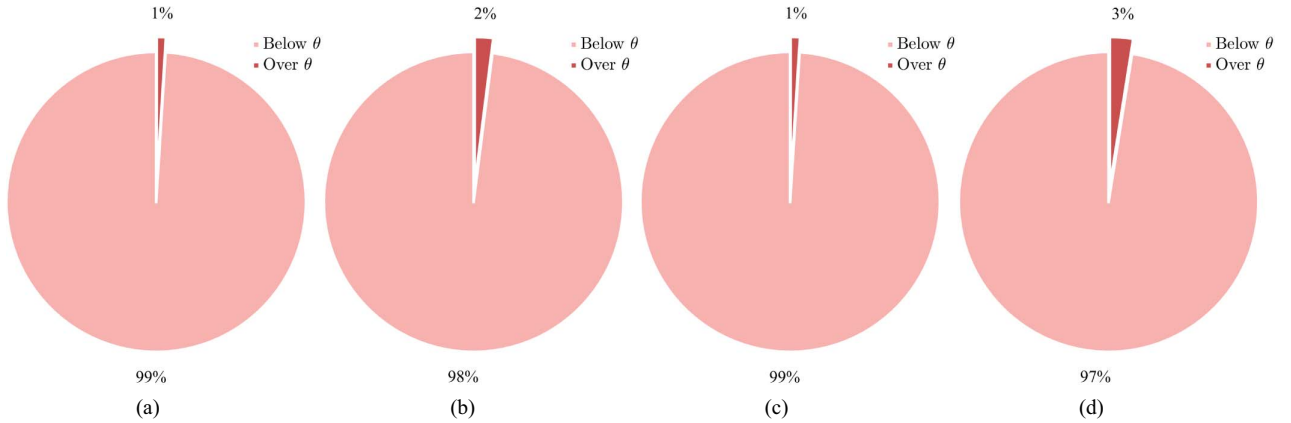
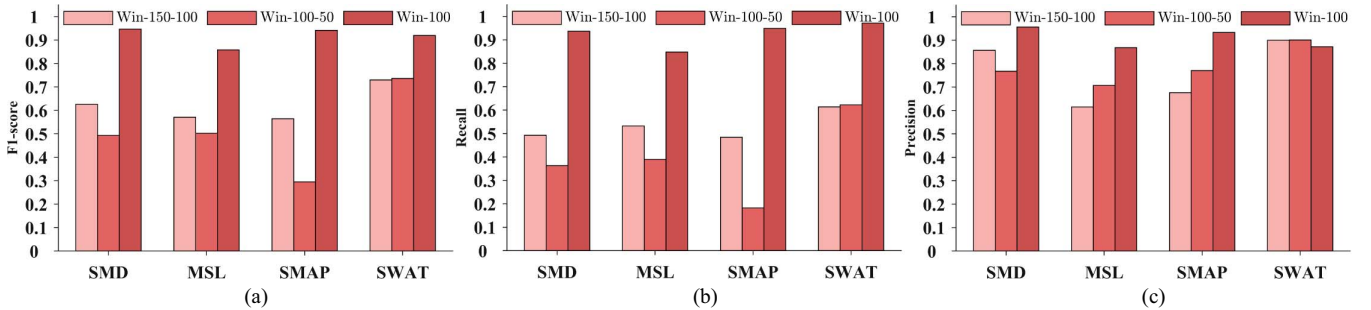
Fig. 13. Sample proportion by the best θ about imputation error. Our final classifier confirms the anomaly according to the best threshold θ . the predicted anomaly ratios are fewer than the actual anomaly ratios. More anomaly time stamps are identified as normal ones. To some extent, our LLM-AD depends on detecting the part of the anomalous time stamp and then confirm the whole anomalies in the same window. (a) SMD. (b) MSL. (c) SMAP. (d) SWAT.

Fig. 14. Performance comparison of nonoverlapping window with overlapping window on LLM-AD. (a) F1-score. (b) Recall. (c) Precision.

V. CONCLUSION

In this article, we propose LLM-AD, an LLM-based framework for the multivariate time series anomaly detection task. The LLM-AD innovates the unsupervised textless prompt by the masked operation and fine-tunes the LLMs to fill the masked variates. Through extensive experiments, we verify the effectiveness of our LLM-AD across several baseline algorithms. Moreover, our research work demonstrates that the text-based general LLMs can still fulfil higher achievement for one special time series task.

Furthermore, we opened the door to applying LLMs to anomaly detection. The image and text are also powerful data

sources that assist in the time series analysis. Our future work will include but not be limited to LLM-based multimodal anomaly detection.

REFERENCES

- [1] C. Zhang et al., "A deep neural network for unsupervised anomaly detection and diagnosis in multivariate time series data," in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 1409–1416, doi: 10.1609/AAAI.V33I01.33011409.
- [2] Q. Xie, P. Zhang, B. Yu, and J. Choi, "Semisupervised training of deep generative models for high-dimensional anomaly detection," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 6, pp. 2444–2453, Jun. 2022, doi: 10.1109/TNNLS.2021.3095150.
- [3] Y. Li, X. Peng, J. Zhang, Z. Li, and M. Wen, "DCT-GAN: Dilated convolutional transformer-based GAN for time series anomaly detection,"

- IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 4, pp. 3632–3644, Apr. 2023, doi: 10.1109/TKDE.2021.3130234.
- [4] Y. Yang, C. Zhang, T. Zhou, Q. Wen, and L. Sun, “DCdetector: Dual attention contrastive representation learning for time series anomaly detection,” in *Proc. 29th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, 2023, pp. 3033–3045, doi: 10.1145/3580305.3599295.
 - [5] Y. Su, Y. Zhao, C. Niu, R. Liu, W. Sun, and D. Pei, “Robust anomaly detection for multivariate time series through stochastic recurrent neural network,” in *Proc. 25th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, 2019, pp. 2828–2837, doi: 10.1145/3292500.3330672.
 - [6] K. Hundman, V. Constantinou, C. Laporte, I. Colwell, and T. Söderström, “Detecting spacecraft anomalies using LSTMs and non-parametric dynamic thresholding,” in *Proc. 24th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, 2018, pp. 387–395, doi: 10.1145/3219819.3219845.
 - [7] A. P. Mathur and N. O. Tippenhauer, “Swat: A water treatment testbed for research and training on ICS security,” in *Proc. CysWater*, 2016, pp. 31–36.
 - [8] L. Xu, Z. Han, Z. Wang, and D. Zhao, “Finding component relationships: A deep-learning-based anomaly detection interpreter,” *IEEE Trans. Computat. Social Syst.*, vol. 11, no. 3, pp. 4149–4162, Jun. 2024, doi: 10.1109/TCSS.2024.3360435.
 - [9] S. Lin, R. Clark, R. Birke, S. Schönborn, N. Trigoni, and S. J. Roberts, “Anomaly detection for time series using VAE-LSTM hybrid model,” in *IEEE Int. Conf. Acoust., Speech Signal Process.*, 2020, pp. 4322–4326, doi: 10.1109/ICASSP40776.2020.9053558.
 - [10] Y. Zhang, Y. Chen, J. Wang, and Z. Pan, “Unsupervised deep anomaly detection for multi-sensor time-series signals,” *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 2, pp. 2118–2132, Feb. 2023, doi: 10.1109/TKDE.2021.3102110.
 - [11] A. Deng and B. Hooi, “Graph neural network-based anomaly detection in multivariate time series,” in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 4027–4035, doi: 10.1609/AAAI.V35I5.16523.
 - [12] W. Zhang, C. Zhang, and F. Tsung, “Grelen: Multivariate time series anomaly detection from the perspective of graph relational learning,” in *Proc. Int. Joint Conf. Artif. Intell.*, 2022, pp. 2390–2397, doi: 10.24963/IJCAI.2022/332.
 - [13] S. Tuli, G. Casale, and N. R. Jennings, “TranAD: Deep transformer networks for anomaly detection in multivariate time series data,” *Proc. VLDB*, vol. 15, no. 6, pp. 1201–1214, 2022, doi: 10.14778/3514061.3514067.
 - [14] J. Xu, H. Wu, J. Wang, and M. Long, “Anomaly transformer: Time series anomaly detection with association discrepancy,” in *Proc. Int. Conf. Learn. Representations*, 2022, pp. 5203–5222.
 - [15] C. Raffel et al., “Exploring the limits of transfer learning with a unified text-to-text transformer,” *J. Mach. Learn. Res.*, vol. 21, no. 140, pp. 1–67, 2020.
 - [16] T. B. Brown et al., “Language models are few-shot learners,” in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 1877–1901.
 - [17] H. Touvron et al., “LLaMA: Open and efficient foundation language models,” 2023, *arXiv:2302.13971*.
 - [18] C. Chang, W.-C. Peng, and T.-F. Chen, “LLM4TS: Aligning pre-trained LLMs as data-efficient time-series forecasters,” *ACM Trans. Intell. Syst. Technol.*, vol. 16, no. 3, pp. 60:1–60:20, 2025, doi: 10.1145/3719207.
 - [19] S. Q. Nate Gruver, M. Finzi, and A. G. Wilson, “Large language models are zero shot time series forecasters,” in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36, 2023, pp. 19622–19635.
 - [20] H. Xue and F. D. Salim, “PromptCast: A new prompt-based learning paradigm for time series forecasting,” *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 11, pp. 6851–6864, Nov. 2024, doi: 10.1109/TKDE.2023.3342137.
 - [21] D. Cao et al., “TEMPO: Prompt-based generative pre-trained transformer for time series forecasting,” in *Proc. Int. Conf. Learn. Representations*, 2024, pp. 54813–54845.
 - [22] M. Jin et al., “Time-LLM: Time series forecasting by reprogramming large language models,” in *Proc. Int. Conf. Learn. Representations*, 2024, pp. 27928–27951.
 - [23] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, and G. Neubig, “Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing,” *ACM Comput. Surv.*, vol. 55, no. 9, pp. 195:1–195:35, 2023, doi: 10.1145/3560815.
 - [24] G. Qin and J. Eisner, “Learning how to ask: Querying LMs with mixtures of soft prompts,” in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics: Human Lang. Technol.*, 2021, pp. 5203–5212, doi: 10.18653/V1/2021.NAACL-MAIN.410.
 - [25] B. Lester, R. Al-Rfou, and N. Constant, “The power of scale for parameter-efficient prompt tuning,” in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2021, pp. 3045–3059, doi: 10.18653/V1/2021.EMNLP-MAIN.243.
 - [26] H. Liu, Z. Zhao, J. Wang, H. Kamarthi, and B. A. Prakash, “LSTPrompt: Large language models as zero-shot time series forecasters by long-short-term prompting,” in *Findings Assoc. Comput. Linguistics*, 2024, pp. 7832–7840.
 - [27] J. Wei et al., “Chain-of-thought prompting elicits reasoning in large language models,” in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 24824–24837.
 - [28] C. Sun, Y. Li, H. Li, and S. Hong, “TEST: Text prototype aligned embedding to activate LLM’s ability for time series,” in *Proc. Int. Conf. Learn. Representations*, 2024.
 - [29] Z. Wang et al., “Learning to prompt for continual learning,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 139–149, doi: 10.1109/CVPR52688.2022.00024.
 - [30] F. Jia, K. Wang, Y. Zheng, D. Cao, and Y. Liu, “GPT4MTS: Prompt-based large language model for multimodal time-series forecasting,” in *Proc. AAAI Conf. Artif. Intell.*, 2024, pp. 23343–23351, doi: 10.1609/AAAI.V38I21.30383.
 - [31] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, and Bert, Pre-Training, “Deep bidirectional transformers language understanding,” in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics: Human Lang. Technol.*, 2019, pp. 4171–4186, doi: 10.18653/V1/N19-1423.
 - [32] A. Vaswani et al., “Attention is all you need,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 5998–6008.
 - [33] K. Rasul, C. Seward, I. Schuster, and R. Vollgraf, “Autoregressive denoising diffusion models for multivariate probabilistic time series forecasting,” in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 8857–8868.
 - [34] Y. Liang, et al., “Foundation models for time series analysis: A tutorial and survey,” in *Proc. 30th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, 2024, pp. 6555–6565, doi: 10.1145/3637528.3671451.
 - [35] C. Wang et al., “Drift doesn’t matter: Dynamic decomposition with diffusion reconstruction for unstable multivariate time series anomaly detection,” in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36, 2023, pp. 10758–10774.
 - [36] H. Wu, T. Hu, Y. Liu, H. Zhou, J. Wang, and M. Long, “TimesNet: Temporal 2D-variation modeling for general time series analysis,” in *Proc. Int. Conf. Learn. Representations*, 2023, pp. 6423–6445.
 - [37] L. Donghao and W. Xue, “ModernTCN: A modern pure convolution structure for general time series analysis,” in *Proc. Int. Conf. Learn. Representations*, 2024, pp. 34209–34251.
 - [38] X. W. L. S. R. J. Tian Zhou and P. Niu, “One fits all: Power general time series analysis by pretrained LM,” in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36, 2023, pp. 43322–43355.
 - [39] S. Gao, T. Koker, O. Queen, T. Hartvigsen, T. Tsiligkaridis, and M. Zitnik, “UniTS: A unified multi-task time series model,” in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 37, 2024, pp. 140589–140631.
 - [40] Z. He et al., “A spatiotemporal deep learning approach for unsupervised anomaly detection in cloud systems,” *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 4, pp. 1705–1719, Apr. 2023, doi: 10.1109/TNNLS.2020.3027736.
 - [41] Y. Wang, et al., “FreeMatch: Self-adaptive thresholding for semi-supervised learning,” in *Proc. Int. Conf. Learn. Representations*, 2023, pp. 21104–21123.
 - [42] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, “Language models are unsupervised multitask learners,” *OpenAI*, vol. 1, no. 8, p. 9, 2019.
 - [43] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. Int. Conf. Learn. Representations*, 2015, pp. 1–15.
 - [44] G. Zhong, F. Liu, J. Jiang, B. Wang, and C. L. P. Chen, “Refining one-class representation: A unified transformer for unsupervised time-series anomaly detection,” *Inf. Sci.*, vol. 656, 2024, Art. no. 119914, doi: 10.1016/J.INS.2023.119914.
 - [45] T. Zhou, Z. Ma, Q. Wen, X. Wang, L. Sun, and R. Jin, “FEDformer: Frequency enhanced decomposed transformer for long-term series forecasting,” in *Proc. Int. Conf. Mach. Learn.*, vol. 162, 2022, pp. 27268–27286.
 - [46] A. Zeng, M. Chen, L. Zhang, and Q. Xu, “Are transformers effective for time series forecasting?” in *Proc. AAAI Conf. Artif. Intell.*, 2023, pp. 11121–11128, doi: 10.1609/AAAI.V37I9.26317.
 - [47] C. Liu, S. He, Q. Zhou, S. Li, and W. Meng, “Large language model guided knowledge distillation for time series anomaly detection,” in *Proc. AAAI Conf. Artif. Intell.*, 2024, pp. 2162–2170.

- [48] Y. Bian, X. Ju, J. Li, Z. Xu, D. Cheng, and Q. Xu, "Multi-patch prediction: Adapting LLMs for time series representation learning," in *Proc. Int. Conf. Mach. Learn.*, 2024, pp. 3889–3912.
- [49] Y. Jeong, E. Yang, J. H. Ryu, I. Park, and M. Kang, "AnomalyBERT: Self-supervised transformer for time series anomaly detection using data degradation scheme," in *Proc. Int. Conf. Learn. Representations*, 2023, pp. 1–11.



Guoxiang Zhong received the Ph.D. degree in electronic information from the South China University of Technology, Guangzhou, China, in 2024.

He is currently a Postdoctoral Researcher with Pengcheng Laboratory, Shenzhen, China. His research interests include cloud computing, anomaly detection, and data mining.



Jie Liu received the B.S. degree in computer science and technology from Nanjing University of Posts and Telecommunications, Nanjing, China, in 2005, and the M.S. degree in computer science and technology in 2007 from the South China University of Technology, Guangzhou, China, where he is currently working toward the Ph.D. degree in electronic information and experimentalist with the School of Computer Science and Engineering.

His research interests include edge computing and cloud-side collaboration.



Fagui Liu (Member, IEEE) received the M.S. degree from Beihang University, Beijing, China and the Ph.D. degree from the South China University of Technology, in 1991 and 2006, respectively, both in computer science and technology.

She is currently a Professor with the School of Computer Science and Engineering, South China University of Technology. She is also with the Department of New Networks, Peng Cheng Laboratory, Shenzhen, China. Her research interests include service computing, Internet of Things, cloud

computing, and big data.



Jun Jiang (Member, IEEE) received the Ph.D. degree in computer science and technology from the South China University of Technology, Guangzhou, China, in 2022.

He was a Visiting Student with Pengcheng Laboratory, Shenzhen, China, from 2022 to 2023. He worked as a Lecturer with the College of Information Science and Technology and the College of Artificial Intelligence, Nanjing Forestry University, Nanjing, China, from 2023 to 2024. He is currently a Postdoctoral Researcher with the Department of

New Networks, Pengcheng Laboratory. His research interests include data stream classification, anomaly detection, cloud computing, and Internet of Things.



Bin Wang received the B.S. and Ph.D. degrees in computer science and technology from the South China University of Technology, in 2014 and 2021, respectively.

He is currently a Research Scientist with the Department of New Networks, Pengcheng Laboratory, Shenzhen, China. His research interests include cloud computing, edge computing, and energy efficiency.



Fan Zhang received the Ph.D. degree in computer science and technology from the Northwestern Polytechnical University, Xi'an, China, in 2009.

He is currently a Engineering Researcher with the Department of New Networks, Pengcheng Laboratory, Shenzhen, China. His research interests include industrial internet and data mining.



C. L. Philip Chen (Life Fellow, IEEE) received the M.S. degree in electrical engineering from the University of Michigan at Ann Arbor, Ann Arbor, MI, USA, in 1985, and the Ph.D. degree in electrical engineering from Purdue University, West Lafayette, IN, USA, in 1988.

He was a Tenured Professor with the Department Head and an Associate Dean of two different universities in the U.S. for 23 years. He is currently the Head with the School of Computer Science and Engineering, South China University of Technology,

Guangzhou, Guangdong, China. His current research interests include systems, cybernetics, and computational intelligence. He is a fellow of AAAS, IAPR, CAA, and HKIE.

Dr. Chen received the 2016 Outstanding Electrical and Computer Engineers Award from his alma mater, Purdue University. He has been the Editor-in-Chief of IEEE Transaction on Systems, MAN, AND CYBERNETICS: SYSTEMS since 2014 and an Associate Editor of several IEEE Transactions. He was the Chair of TC 9.1 Economic and Business Systems of the International Federation of Automatic Control from 2015 to 2017 and also a Program Evaluator of the Accreditation Board of Engineering and Technology Education of the U.S. for computer engineering, electrical engineering, and software engineering programs. He was the IEEE SMC Society President from 2012 to 2013 and a Vice President of the Chinese Association of Automation (CAA).